Angular – client base framework used in creating web applications and often used in building single page applications. It follows **Component-Based Architecture:**

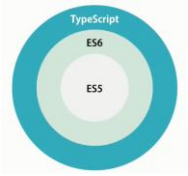
- Angular promotes building applications as a tree of components. Each component manages its own view and logic.
- This architecture promotes modularity, reusability, testability, and maintainability.
- Components communicate with each other through input and output properties (@Input, @Output) or by using shared services
- *What are Angular components, modules, and standalone components?*
    - Angular components are reusable UI units with HTML templates and TypeScript logic.
    - Modules (@NgModule) traditionally group related components/services, but Angular 19 prioritizes standalone components, which don't require NgModules. With standalone: true now the default, new projects can define components, directives, and pipes without modules,
    - NgModules still exist for compatibility, but new projects can often skip them.

Single page applications – web applications that loads single html page. It dynamically update the current page rather than loading entire new pages.

Typescript – typed programming language and a superset (Every JavaScript code is a valid typescript code) of JavaScript that compiles to plain JavaScript and with added features. Syntax based on ES6.

Generics in TypeScript enable functions or classes to work with multiple types while maintaining type safety. They improve reusability, preserve type information, and help the compiler catch type errors/ mismatches. This avoids using any and losing type information.

- . For example, function identity<T>(arg: T): T { return arg; } works for any type and returns the same type.



Es6 features

Block scoped Let const
Arrow functions (fat arrow)– ex. use in map. forEach, anything that deals with list
Export and import keywords – modules designed around
Classes – java like borrows the concept of classes
Template literal - ${} clean way to create strings
Spread operator(…) -= copying items in array and add a new one.

Angular JS	***Angular***
Javascript based framework	Typescript based Framework
Doesn't support concept of dependency injection	Supports concept of dependency injection

jQuery vs angular – no two-way binding, no form validation, no support for restful api.

AOT (ahead of time compilation)

- compiles the application at build time.
- recompiles the applications components and their templates during build process.
- It is the default for production builds.
- compiles your Angular templates (HTML) and components (TypeScript code) into plain JavaScript and HTML during build time rather than runtime. *Compiled before* the browser downloads and runs the code.

- faster rendering (browser runs pre-compiled code), smaller application size (compiler removes unused parts), and earlier detection of template errors during the build process.

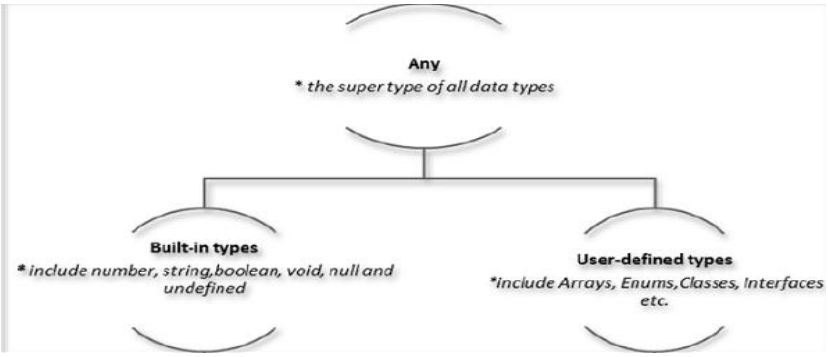
JIT (just in time compilation)

- Compiles the application at runtime.
- Compiles the code into machine language at runtime, rather than before execution.
- It is used during development.
- compilation happens in the browser at runtime (used during development).

Angular benefits – single page application, two way data binding, cross platform app development, animation support, angular cli, custom directives. (stcaac)

Disadvantages – complex SPA, learning curve (cl)

Data types



any - we want to opt-out of type-checking and let the values pass through compile-time checks

let - allows you to declare variables that are limited in scope to the block, statement, or expression where they are defined/used. In contrast, var declares variables with function scope, meaning they are accessible throughout the entire function regardless of block boundaries.s

Building blocks of an angular

---

1. Components
2. Directives
3. Services
4. Data Binding
5. Modules
6. Routing
7. Template
8. Dependency Injection
9. Metadata

Component – is the building block of an angular application to which an angular application is made up of. Angular is essentially a hierarchy of component. It encapsulate the view (HTML), logic (TypeScript), and styling (CSS/SCSS) into self-contained units, facilitating reusability.

```
@Component({
  selector: 'new-recipe',
  templateUrl: './new-recipe.component.html',
```

```
  styleUrls: ['./new-recipe.component.scss'],
  providers: [NgbModal]
})
```

Directive – instructions which tells angular to do something.

```
@Directive({
  selector: '[myDirective]'
})
export class MyDirective {
```

There are three kinds of directives in Angular:

- **Components**—directives with a template. The most common of the three directives.
- **Attribute directives**—change the appearance or behavior of an element, component, or another directive. E.g: ngClass, ngStyle
- **Structural directives**—change the DOM layout by adding and removing DOM elements. Used to change the structure of the view. E.g: *ngFor, *ngIf,

1. Explain the new **built-in Control Flow syntax (**@if**,** @for**,** @switch**) in Angular templates.**
    - Streamlines conditional (@if), list (@for), and switch (@switch) rendering in Angular templates, eliminating the need for structural directives like *ngIf, *ngFor, and *ngSwitch.
    - Enhances readability, improves type checking, and may boost build performance by eliminating reliance on directive sugar syntax.

Data Binding - is the synchronization between the model and the view (between the component class and its template)

- ***One way binding***

  Data Flow INTO the View

  String Interpolation – syntax use to display component data to the template

  E.g : {{title}}

  Property Binding

  E.g : <input [required]="isRequired">

  <input [value]="myString">

  Data Flow OUT of the View

  Event Binding

  E.g : <button (click)="changeTitle()".>

  - the component class (ts) will handle the event

- ***Two-Way Binding –***
  - Accomplished using the angular directive ngModel and two-way binding syntax. ngModel – angular directive used to get, set or change the value of form controls (input, checkbox,radio etc.)
  - allows data to flow between a component property and a form control (or another input) in both directions. This means changes in the UI update the model, and changes in the model update the UI.

Pipes - Used to transform how your data will be display in the view.

1. **How do you create and use custom pipes in Angular 19?** - They are implemented by creating a class annotated with @Pipe and implementing a transform() method, allowing you to encapsulate transformation logic that can be reused throughout an application.

```
@Pipe({
 name: 'customPipe'
})
```

DecimalPipe- {{ 1.2548 | number: '1.2-2' }}
PercentPipe- {{ 0.25 | percent }}
UpperCasePipe and LowecasePipe- {{ Angular | uppercase }} {{ Angular | lowercase }}
SlicePipe - {{ Angular | slice: 0: 3}}

Services – SCUMP

- Singleton object instantiated only once during lifetime of an application.
- Contains methods that maintain the data throughout the life of an application.
- Use to organize and shared business logic , data and functions (utility functions) with various components of an angular application.
- Marked with the @Injectable() decorator, making them available for injection into other parts of the application via Dependency Injection.
- Promote separating concerns from components.

```
@Injectable({
  providedIn: 'root'
})
export class MyService {
@Injectable({
 providedIn: 'root',
})
export class AbsenceService {
 constructor(private http: HttpClient) {}

 saveAbsences(saveAbsencesRequest: SaveAbsencesRequest): Observable<SaveAbsencesResponse> {
  return this.http.post(`${env.apiUrl}/absence`, saveAbsencesRequest) as Observable<
   SaveAbsencesResponse
  >;
 }
}
```

HttpClient inject in a constructor of a service to perform http request

Modules – (GHW)

- group components, directives, pipes and services that are related, in such a way that can be combined with other modules to create an application.
- help in lazy loading features, reducing the initial bundle size, and facilitate better separation of concerns. (lrf)
- With Angular 19, module organization becomes even more advantageous as the applications scale.

```
@NgModule({
 declarations: [
   //components,
```

```
 ],
 imports: [
  //other modules,
 ],
 providers: [
  //services,
 ],
 bootstrap: [AppComponent]
})
export class AppModule { }
```

Routing - enables navigation from one view to the next.

The Angular Router

- comprised of path (refers to the part of the URL that determines a unique view) and corresponding component to display.
- supports features like lazy loading, route guards, and parameterized routes.

```
const routes: Routes = [
 { path: '', redirectTo: RouteConstants.ADMIN_APPROVE_NEW_RECIPE, pathMatch: 'full' },
 {
  path: RouteConstants.ADMIN_APPROVE_NEW_RECIPE,
  component: ApproveNewRecipeComponent,
  data: { roles:[RoleConstants.PHARMACY_TECH, RoleConstants.INTERN, RoleConstants.PHARMACY_MANAGER,RoleConstants.PHARM
ACIST], pageTitle: 'Approve New Recipe' },
  canActivate: [AppGuardService]
 }
];
```

```
const routes: Routes = [
 { path: '', redirectTo: RouteConstants.LOGIN, pathMatch: 'full'},
 { path: RouteConstants.LOGIN, loadChildren: './login/login.module#LoginModule'},
```

Lazy Loading – loading on demand. Loading what essentials and needed.
Module path route you want to lazyload use loadChildren

```
const routes: Routes = [
 {
  path: '',
  loadChildren: () => import('@modules/home/home.module').then((m) => m.HomeModule),
  canActivate: [AuthGuard],
 },
 {
  path: 'session',
  loadChildren: () => import('@modules/session/session.module').then((m) => m.SessionModule),
  canActivate: [UnauthGuard],
 },
];
@NgModule({
 imports: [RouterModule.forRoot(routes, { onSameUrlNavigation: 'reload' })],
 exports: [RouterModule],
```

```
})
export class AppRoutingModule {}
```

Template - is an HTML snippet that tells **Angular** how to render the component. Basically a component's view.

Template-Driven Forms: (upb)

- Use directives like ngModel to manage Form logic within the template.
- Provide a simpler, straightforward, less programmatic approach suited for basic forms with minimal logic and minimal coding.
- Best for forms with minimal validation but less scalable for complex applications.

Reactive Forms: (usop)

- Use FormControl, FormGroup, and FormArray in the component class to define form behavior and validation programmatically.
- Structure and logic are handled in TypeScript.
- Offer more control, making them ideal for complex forms with dynamic validation and conditional logic.
- Preferred approach for most applications.

- How do you handle validation in Reactive Forms? –
    - In Reactive Forms, validation is defined in the component class by passing validator functions (e.g., Validators.required, Validators.email, or custom validators) as the second argument in the FormControl constructor.
    - Control's validation status (valid, invalid, touched, dirty, errors) can be used in the template to show error messages conditionally.

Dependency Injection (DI) in allows components and services to receive dependencies from an external source rather than creating them themselves. (upid)

- Uses providers (marked with @Injectable) to manage dependency instances.
- Promotes loose coupling and code reusability.

- Injecting dependencies via a constructor allows Angular to provide required instances.
- Declaring @Injectable({providedIn: 'root'}) ensures a singleton service, making it accessible across the app for shared functionality like data handling and logging.

Ways to share data between components in Angular: (ISRLR)

- **Input/Output Properties (**@Input**,** @Output**):** For parent-child communication. Parent passes data down via @Input, child emits events up via @Output and EventEmitter.

- **Shared Service:** For communication between components that don't have a direct parent-child relationship (e.g., siblings, unrelated components). Components inject the shared service and use it to store and access shared data, often using Observables or Signals within the service.

- **RxJS Subject/BehaviorSubject in a Service:** A powerful way to share streams of data reactively.

- **Local Storage/Session Storage:** For sharing simple data across browser sessions (less common for in-app component communication).

- **Routing (Route Parameters, Query Parameters):** Passing data during navigation.

Angular LifeCycle Hooks (mgm)

- methods called by Angular at specific points of a component's life
- give developers control over a component's evolution - from creation to destruction.
- methods like ngOnInit, ngOnChanges, ngAfterViewInit, and ngOnDestroy allow you to execute initialization logic, respond to input changes, handle view-related tasks, and perform cleanup operations in Angular components.

ngOnChanges() –

Called once during the component's creation and whenever an input-bound property changes.

- Received a SimpleChanges object as a parameter which contains information of input properties that has changed.

ngOnInit() –

Called once after the first ngOnChanges.
Runs after the constructor and once component inputs are initialized.

- Used for setting up necessary data (data fetching), setting up to subscriptions (subscribing to services), and handling initialization logic.

ngDoCheck() –

Called immediately after ngOnChanges and ngOnInit.
Runs during every change detection run.

- Used to manually detect and respond to changes that Angular's default change detection might miss, or Angular can't or won't detect on its own.
- Useful when you need to handle complex changes or custom behavior in your component.

ngAfterContentInit() –

Called once after the first ngDoCheck,
Running after Angular projects external content into the component's view.

- Useful for performing any additional initialization tasks required for content that has been projected into the component (content children).

```
import { Component, AfterContentInit, ContentChild } from '@angular/core';

@Component({
 selector: 'app-parent',
 template: `
   <ng-content></ng-content>
 `
})
export class ParentComponent implements AfterContentInit {
 @ContentChild('projectedContent') content: any;
 ngAfterContentInit() {
   console.log('Content initialized:', this.content);
 }
}
```

```
@Component({
 selector: 'app-child',
 template: `
  <app-parent>
    <p #projectedContent>Projected Content</p>
  </app-parent>
 `
})
export class ChildComponent {}
```

ngAfterContentChecked() –

Called after ngAfterContentInit and every subsequent ngDoCheck.
Runs after Angular checks the content projected into the component's2 view.

- Allows to react to any changes made to the content that has been projected into your component, ensuring that your component can handle changes dynamically.

ngAfterViewInit() –

Called once after the first ngAfterContentChecked.
Runs after Angular initializes the component's views and child views.

- Only invoked once after the view has been fully initialized, making it the perfect place to perform any additional setup or initialization required for your component's view.

```
@Component({
 selector: 'app-view-child',
 template: `
  <p #viewChildElement>View Child Element</p>`
})
export class ViewChildComponent implements AfterViewInit {
 @ViewChild('viewChildElement') viewChildElement: any;
 ngAfterViewInit() {
  console.log('View initialized:', this.viewChildElement.nativeElement.textContent);
 }
}
```

ngAfterViewChecked() –

Called after ngAfterViewInit and every subsequent ngAfterContentChecked.
Run after Angular checks the component's views and child views.

- Used to act upon any changes after the view has been checked.

ngOnDestroy() –

Called just before Angular destroys the component or before component is removed from the DOM.

- Used for cleanup (unsubscribing from Observables, detaching event listeners) to avoid memory leaks.

---

ngOnChanges() responds and is triggered only when input properties values change. In contrast, ngDoCheck() provides finer control, running during every change detection cycle, regardless of whether input properties have changed.

Angular used npm (node package manager to get the libraries and dependencies needed in your application).

Angular Cli commands

- `npm install`
- `npm start/ng serve`
- `ng new myapp`
- `ng generate component mycomponent/ng generate component myfolder/mycomponent`
- `ng build --configuration=production`

package.json – dependencies

angular-cli.json - file for index.html, declared styles and scripts needed to use the dependencies, environments (prod or dev, points to files) (ISE)

app.module.ts – imnport all modules of the applications. And third party modules needed in the entire application,

app component

under app component

<router-outlet></router-outlet>

---

ng-content is a Angular element used for **content projection**. It allows you to insert or project content from a parent component into a child componen

```
// parent.component.html
<app-child>
 <p>This content will be projected into the child component.</p>
</app-child>
```

```
// child.component.html
<div>
 <ng-content></ng-content>
</div>
```

ng-container is an Angular element that allows you to group elements without adding any extra tags to the DOM. It's useful when you need a wrapper element for structural directives like ngIf or ngFor, but don't want to introduce additional HTML elements in your final rendered output.

```
// component.html
<ng-container *ngIf="showContent">
  <div>
    <p>This content will be displayed if showContent is true.</p>
  </div>
</ng-container>
```

ng-template is an Angular element used for rendering HTML templates. The content within an ng-template tag is not rendered by default. Instead, it's a placeholder for rendering embedded views dynamically based on conditions or structural directives.

```
<!-- component.html -->
<div *ngIf="isLoading; else content">
 Loading...
</div>
<ng-template #content>
 <p>The actual content goes here once loading is complete.</p>
</ng-template>
```

@ViewChild('variableAttchedToChilden') is a decorator in Angular used to access a child component, directive, or DOM element from within a parent component. It allows you to query and interact with the component or element in your TypeScript code.

```
@Component({
 selector: 'app-parent',
 template: '<input #myInput type="text" />'})
export class ParentComponent {
 @ViewChild('myInput') 'myInput'!: ElementRef;

 ngAfterViewInit() {
  this.myInput.nativeElement.focus();
 }}
```

? – question Mark/Elvis/Safe Navigation Operator – nullity check

---

Observable provides a stream of values, lazy, cancellable and have a rich set of operators. are widely used for forms, HTTP, routing, and handling real-time events efficiently.

In Angular, HttpClient.get() returns an Observable, allowing multiple emissions and unsubscription.

Promise can provide a single value, eager, not cancellable,

Describe the Angular change detection mechanism. What is the difference between the Default and OnPush strategies?

- **Angular's change detection** automatically checks if the application state has changed and updates the UI accordingly.

- **Default Strategy (CheckAlways)**: *(RC)*
  - Runs frequently (every event, timer, etc.).
  - Checks all components from the root down on any change which can impact performance.

- **OnPush strategy**: *(OR)*
  - Optimizes performance by checking only when: (IOCA)
    - @Input references change
    - Observable emits
    - ChangeDetectorRef.detectChanges() is called
    - Async pipe emits a new value.
  - Reduces unnecessary or redundant checks for immutable data, boosting performance, making it ideal for performance-sensitive apps.

Angular Testing - Command -n test

```
Describe('ComponentName'
It( 'message', ()=> {
  Expect(value from component).toBe();
});
```